

# CITYPULSE

## Complete Hackathon Project Plan + Market Research + Antigravity Build Specification

*AmiHacks • Problem Statement 2 • The Live Civic Health Dashboard*

### 1. Executive Summary

CityPulse should be designed as a civic intelligence platform, not simply as a dashboard. It will ingest heterogeneous civic signals such as weather, AQI, traffic, public transit, flooding, noise, power outages, gas incidents and citizen complaints; normalize them into a common spatial-temporal model; detect anomalies and possible relationships; forecast selected risks; and present grounded explanations to citizens and city operators.

The official problem statement requires at least three distinct data types, normalization and timestamps, basic anomaly/correlation detection, a live glanceable dashboard or map, and a plain-language summary. It explicitly permits public or synthetic data and recommends graceful degradation, privacy protection, and careful wording of correlations as possible links rather than confirmed causes.

### 2. Source Material Reviewed

- AmiHacks Problem Statement -2.docx: CityPulse, The Live Civic Health Dashboard.
- The related AmiHacks Problem Statement - 3 document was also surfaced in the project search, but it describes SentinelAPI and is not the basis for CityPulse requirements.
- The CityPulse folder itself was not surfaced as a separate Library folder or collection in the available file index. Therefore, this plan uses the CityPulse problem statement plus the additional architecture/features supplied in the conversation, and does not pretend unseen project files were reviewed.

The CityPulse statement identifies the core challenge as fusing scattered civic feeds with different formats, timestamps and update rates into a single trustworthy pulse that a non-technical person can understand quickly.

### 3. Problem Statement in Simple Terms

Today, a resident may need separate apps for weather, traffic, transit, pollution and civic complaints. City staff similarly see data in departmental silos. CityPulse creates one shared operational picture.

#### DATA SOURCES

Weather | AQI | Traffic | Transit | Flooding | Noise | Power | Gas | Complaints



INGESTION LAYER

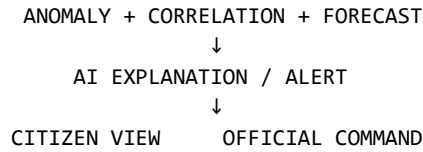


NORMALIZATION LAYER



CITYPULSE DATA





## 4. Product Vision

Positioning:

**“CityPulse is a live civic intelligence layer that turns disconnected city signals into understandable, location-aware and time-aware insights.”**

Core loop:

OBSERVE → NORMALIZE → CONNECT → DETECT → PREDICT → EXPLAIN → ALERT/ACT → LEARN

The platform should answer five questions:

1. What is happening right now?
2. Where is it happening?
3. Which independent signals are changing together?
4. What might happen next?
5. Who needs to know or act?

## 5. Market Research: Existing Products and What They Already Do

The market is not empty. Several products already cover parts of the CityPulse concept. The project should therefore differentiate through the combination and workflow rather than claiming that no competitor exists.

| Product / Platform             | What it does                                                                                                | Overlap with CityPulse                                    | Gap / opportunity for our prototype                                                                                                                                  |
|--------------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UrbanLogiq                     | Government-focused data integration, normalization, AI/ML analytics and cross-domain decision intelligence. | Very high: data fusion, government analytics, prediction. | Build a hackathon-sized, citizen + official experience with a transparent event timeline, evidence trail and rapid multi-signal correlation demo.                    |
| ArcGIS Hub / ArcGIS Dashboards | Open-data sharing, maps, dashboards, APIs, collaboration and real-time location-based analytics.            | Very high: maps, civic data, public dashboards.           | Focus on an intelligence layer that automatically detects cross-domain events and explains why signals are connected, instead of primarily publishing datasets/maps. |
| Waze for Cities                | Real-time traffic incidents, road closures, hazards, traffic                                                | High for mobility and incidents.                          | CityPulse should not be a traffic-only product. Combine mobility with                                                                                                |

|                               |                                                                                                          |                                          |                                                                                                                              |
|-------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
|                               | management and city-to-driver communication.                                                             |                                          | weather, AQI, flooding, power, noise, transit and citizen reports.                                                           |
| urbanOS CityPulse             | Live visitor analytics, forecasts, noise/loudness, comparisons, map views and an automatic insight feed. | Name overlap and some analytics overlap. | Use a clearly different brand/product framing and focus on multi-domain civic health rather than footfall/visitor analytics. |
| Socrata / Open Data platforms | Government data catalogs, APIs, querying and exports.                                                    | High at the data-access layer.           | CityPulse sits above data catalogs and turns heterogeneous datasets into a live operational narrative and event graph.       |

Research sources:

- UrbanLogiq describes a unified government data platform with data cleaning/centralization, insight automation, analytics and prediction.
- ArcGIS Hub and ArcGIS Dashboards provide open-data discovery, APIs, maps, dashboards and real-time location-based analytics.
- Waze for Cities provides real-time traffic incidents, closures, hazards, traffic monitoring and city communication.
- urbanOS CityPulse provides live visitor analytics, forecasts, loudness analytics, comparison views and an insight feed.
- Socrata provides government open-data APIs and exports.

## 6. How CityPulse Should Be Different

Do not compete by saying “we have a map.” The market already has maps. Differentiate with the intelligence loop.

**Multi-signal event fusion:** Detect an event from several independent signals in the same area/time window: e.g., heavy rain + traffic slowdown + flood complaints + transit delays.

**Evidence-first explanations:** Every insight shows the signals, timestamps, locations, source types and changes that produced it.

**Citizen + command center in one system:** Citizens get simple information; officials get operational detail, prioritization and response workflows.

**Live civic health as a dynamic state:** Show not just raw metrics but what is changing and which factors are driving the change.

**Explainable risk forecasting:** Forecast selected risks and show the input factors rather than producing a black-box prediction only.

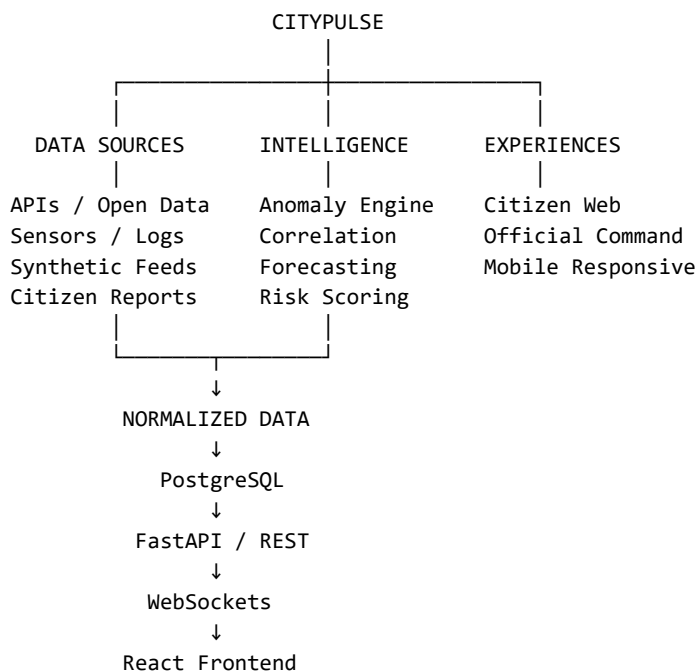
**Historical replay:** Replay a multi-hour or multi-day event to demonstrate how the platform detected a developing situation.

**Data provenance:** Every value carries source type, last-updated time, freshness and confidence metadata.

**Graceful degradation:** If one feed fails, CityPulse keeps operating and clearly marks the missing/stale feed.

**India-first hackathon practicality:** Use Indian city/state hierarchy, synthetic feeds where live municipal data is unavailable, and a design that can expand city by city.

## 7. Recommended System Architecture



Recommended MVP stack:

- Frontend: React + Vite + Tailwind CSS + Recharts + a map library.
- Backend: Python + FastAPI.
- Database: PostgreSQL. SQLite is acceptable for the first local prototype.
- Data processing: Pandas/NumPy.
- Analytics: rule-based anomaly/correlation first, Scikit-learn for selected ML features.
- AI: LLM used only for grounded summarization, explanation and optional analyst assistance.
- Real-time: WebSockets for dashboard updates; scheduled ingestion jobs for a 24-hour MVP.
- Kafka: optional advanced layer, not required for the first working demo.
- Google Drive: historical CSV/JSON archive and dataset exchange, not the primary transactional database.

## 8. Complete Frontend File Plan

```
frontend/
├── src/
│   ├── app/
│   │   ├── router.jsx
│   │   ├── providers.jsx
│   │   └── routes.jsx
│   └── pages/
│       ├── LandingPage.jsx
│       ├── PublicDashboard.jsx
│       └── LiveCity.jsx
```

```

├── Neighborhood.jsx
├── Alerts.jsx
├── Incidents.jsx
├── Reports.jsx
├── AdminCommandCenter.jsx
├── Operations.jsx
├── Forecasts.jsx
├── Analytics.jsx
├── HistoricalReplay.jsx
├── DataSources.jsx
├── Settings.jsx
├── components/
│   ├── layout/
│   ├── map/
│   ├── dashboard/
│   ├── incidents/
│   ├── alerts/
│   ├── analytics/
│   ├── forecasts/
│   ├── civic-health/
│   └── common/
├── hooks/
├── services/
│   ├── apiClient.js
│   ├── websocket.js
│   ├── mapService.js
│   └── authService.js
├── store/
├── utils/
├── data/
├── styles/
├── main.jsx
└── public/

```

Important frontend screens:

- Citizen Live Dashboard: civic health, alerts, current events and plain-language summary.
- Interactive Map: layer toggles for weather, AQI, traffic, transit, flooding, noise, power, gas and incidents.
- Event Detail: timeline, supporting signals, evidence, source freshness and confidence.
- Neighborhood View: area-specific conditions and trend changes.
- Official Command Center: active events, priority queue, map, response teams and incident status.
- Analytics: trends, correlations, historical comparison and forecast charts.
- Historical Replay: time slider that replays an event cluster.
- Data Sources: source health, freshness, last update, errors and synthetic/live status.

## 9. Complete Backend File Plan

```

backend/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── dependencies.py
│   └── api/
│       ├── auth.py
│       ├── dashboard.py
│       └── cities.py

```

- ├── events.py
- ├── incidents.py
- ├── alerts.py
- ├── forecasts.py
- ├── analytics.py
- ├── reports.py
- ├── sources.py
- └── citizen\_reports.py
- ├── ingestion/
  - ├── scheduler.py
  - ├── weather.py
  - ├── aqi.py
  - ├── traffic.py
  - ├── transit.py
  - ├── open\_data.py
  - ├── complaints.py
  - ├── synthetic\_feeds.py
  - └── base\_connector.py
- ├── normalization/
  - ├── schema.py
  - ├── location.py
  - ├── timestamps.py
  - └── validators.py
- ├── analytics/
  - ├── anomaly.py
  - ├── correlation.py
  - ├── event\_clustering.py
  - ├── civic\_score.py
  - ├── risk.py
  - └── forecasting.py
- ├── ai/
  - ├── summarizer.py
  - ├── explanation.py
  - └── prompts.py
- ├── services/
  - ├── event\_service.py
  - ├── alert\_service.py
  - ├── report\_service.py
  - └── source\_health.py
- ├── models/
- ├── schemas/
- ├── db/
- ├── utils/
- └── tests/

## 10. API Contract Plan

| Method | Endpoint                        | Purpose                      | MVP?     |
|--------|---------------------------------|------------------------------|----------|
| GET    | /api/v1/dashboard/overview      | Current civic-health summary | YES      |
| GET    | /api/v1/cities                  | City/state list              | YES      |
| GET    | /api/v1/cities/{city_id}/events | Current events for map       | YES      |
| GET    | /api/v1/events/{event_id}       | Event details and evidence   | YES      |
| GET    | /api/v1/incidents               | Incident list                | YES      |
| POST   | /api/v1/reports                 | Citizen issue report         | OPTIONAL |
| GET    | /api/v1/alerts                  | Active alerts                | YES      |

|      |                                     |                               |          |
|------|-------------------------------------|-------------------------------|----------|
| GET  | /api/v1/analytics/trends            | Time-series metrics           | YES      |
| GET  | /api/v1/analytics/correlations      | Detected relationships        | YES      |
| GET  | /api/v1/forecasts                   | Risk/metric forecasts         | ADVANCED |
| GET  | /api/v1/replay/{event_id}           | Historical event replay       | ADVANCED |
| GET  | /api/v1/sources/health              | Feed health/freshness         | YES      |
| POST | /api/v1/admin/incidents/{id}/assign | Assign response resource      | ADVANCED |
| GET  | /api/v1/reports/{id}                | Generate/view incident report | ADVANCED |

## 11. Unified Civic Data Model

Use an event-oriented schema instead of one giant table with dozens of nullable columns.

```
citypulse_events
- event_id
- state
- city
- district
- area
- latitude
- longitude
- timestamp
- category
- metric
- value
- unit
- severity
- status
- source_type
- source_name
- source_id
- freshness_seconds
- confidence
- raw_payload
```

Recommended categories:

- weather
- air\_quality
- traffic
- transit
- flooding
- noise
- power\_outage
- gas\_incident
- citizen\_complaint
- road\_incident

Each observation should be tied to both time and geography so the correlation engine can reason about proximity and time windows.

## 12. Data Ingestion Strategy

Use a hybrid data strategy. The hackathon statement permits public and synthetic data, so do not block the project waiting for every real municipal feed.

| Signal     | Preferred source                  | Fallback                         | Storage                    |
|------------|-----------------------------------|----------------------------------|----------------------------|
| Weather    | Weather API                       | Synthetic weather feed           | events + raw archive       |
| AQI        | Public AQI/API source             | Synthetic AQI                    | events + raw archive       |
| Traffic    | Authorized/public traffic feed    | Synthetic road speeds            | events + road observations |
| Transit    | Public transit feed               | Synthetic GTFS-like feed         | events                     |
| Flooding   | Public incident/open data         | Synthetic citizen reports        | incidents/events           |
| Noise      | Open/sensor data                  | Synthetic sensors                | events                     |
| Power      | Utility/open data where available | Synthetic outage events          | incidents/events           |
| Gas        | Public/open data if available     | Synthetic sensor/incident events | incidents/events           |
| Complaints | Open 311-style data               | Synthetic complaint stream       | events/incidents           |

Use Google Maps/Mapbox for geographic visualization and geocoding where appropriate. Do not treat the map provider as the primary civic database. Store CityPulse observations in your own database.

## 13. Intelligence Layer

### 13.1 Anomaly Detection

- MVP: rolling average + threshold + rate-of-change rules.
- Advanced: Z-score, Isolation Forest or time-series models.
- Always show the baseline used: e.g., current traffic 82% vs typical 45%.

### 13.2 Correlation Engine

```
Possible event:
same geographic zone
+
within time window (e.g. 30 min)
+
related signal categories
+
significant change from baseline
=
candidate multi-signal event
```

Never present correlation as confirmed causation. Use language such as “coinciding with”, “possible relationship”, and “signals overlap in time and area.”

### 13.3 Forecasting

- Predict only a small number of outcomes for the hackathon: traffic congestion, AQI deterioration, or flood-risk level.
- Use interpretable features first.
- Show forecast horizon and confidence/uncertainty.
- Do not present predictions as guarantees.



### 13.4 Civic Health Index

Create a clearly labeled CityPulse-generated composite indicator. Suggested dimensions: Environment, Mobility, Safety, Infrastructure and Public Services. Keep the formula visible in the methodology view.

## 14. AI Layer

- Grounded summary: convert structured current-state data into a short explanation.
- Event explanation: explain which signals changed, where and when.
- Incident report: generate a human-readable event narrative from stored evidence.
- Optional analyst copilot: answer questions such as “What changed in Mansarovar in the last 30 minutes?” using only CityPulse data.
- Do not let the LLM invent sensor values, causes, incidents or official statements.

Recommended pattern:

Structured data → deterministic analytics → evidence bundle → LLM explanation → UI

## 15. User Roles

| Role          | Needs                                                               | Main screens                                        |
|---------------|---------------------------------------------------------------------|-----------------------------------------------------|
| Citizen       | Fast understanding, nearby alerts, map, neighborhood status         | Public Dashboard, Live Map, Alerts, Report Issue    |
| City Operator | Prioritize incidents, monitor feeds, view evidence, assign response | Command Center, Operations, Event Detail, Resources |
| Analyst       | Trends, historical comparisons, data quality, forecasts             | Analytics, Replay, Data Sources                     |
| Admin         | Users, sources, permissions, system health                          | Admin, Sources, Settings                            |

## 16. Website Features That Matter Most

- Live city map with layer controls.
- Civic Health overview with transparent drivers.
- What’s happening now narrative.
- Why is this happening? evidence drawer.
- Event clusters that combine multiple signals.
- Timeline showing signal sequence.
- Source/freshness/confidence indicators.
- Graceful missing-feed handling.
- Citizen report submission.
- Official incident prioritization.
- Historical replay.
- Forecast and risk map.
- Responsive public interface.
- Accessible color/labels, not color-only status.

## 17. Reliability, Privacy and Trust

- Show last-updated time for every feed.
- Show LIVE / PUBLIC / SIMULATED source labels.
- If a feed is stale, do not silently treat it as current.
- Do not identify individual users from complaint/social data.
- Aggregate citizen reports spatially and temporally.
- Maintain source attribution and data lineage.
- Keep an audit log for official actions.
- Use role-based access for administrative functions.
- Make uncertainty visible for forecasts and correlations.

## 18. Antigravity: How to Explain the Project

Do not ask Antigravity to “make a smart city dashboard.” Give it a structured product specification. Use the following prompt as the master instruction after the planning phase.

You are building CityPulse, an advanced Civic Intelligence Platform for the AmiHacks Problem Statement 2: The Live Civic Health Dashboard.

IMPORTANT: Do not build a generic SaaS dashboard. Do not start by inventing random screens. First inspect the existing project structure, routing, theme system, components, data services and API architecture. Then implement the product according to this specification.

### PRODUCT PURPOSE

CityPulse combines weather, AQI, traffic, public transit, flooding, noise, power outages, gas incidents and citizen complaints into a unified time-and-location-aware civic data layer. It detects anomalies and possible relationships between signals, forecasts selected risks, and explains the current situation in plain language.

### CORE LOOP

Observe → Normalize → Connect → Detect → Predict → Explain → Alert/Act → Learn.

### TWO EXPERIENCES

1. Public Citizen Experience
2. City Official Command Center

### PUBLIC PAGES

- Landing
- Live City Dashboard
- Interactive Map
- Neighborhood View
- Alerts
- Incidents
- Reports / Citizen Reporting
- Historical Replay

### OFFICIAL PAGES

- Command Center
- Operations
- Incident Management
- Forecasts
- Analytics
- Resources / Dispatch
- Data Source Health

- Reports
- Settings

#### MAP

Support toggles for:

Weather, AQI, Traffic, Transit, Flooding, Noise, Power, Gas, Complaints and Road Incidents.

Use heatmaps, markers, polygons/zones and route lines where appropriate.

#### CORE DASHBOARD

Show:

- CityPulse civic health indicator
- Environment
- Mobility
- Safety
- Infrastructure
- Public services
- Active alerts
- Top event clusters
- "What's happening now?" narrative
- "Why is this changing?" evidence panel

#### EVENT CLUSTER

An event cluster is created when multiple related signals overlap spatially and temporally.

Example:

heavy rainfall + traffic slowdown + flood complaints + transit delay.

Do NOT state that one event caused another unless the data actually proves it. Use "possible relationship" or "coinciding signals."

#### DATA MODEL

Use an event-oriented normalized schema:

event\_id, state, city, district, area, latitude, longitude, timestamp, category, metric, value, unit, severity, status, source\_type, source\_name, source\_id, freshness\_seconds, confidence, raw\_payload.

#### DATA SOURCES

Support live/public/synthetic data. Every source must show source type and freshness.

The system must continue operating when one source is missing or delayed.

#### BACKEND

Use FastAPI with modular services:

- ingestion
- normalization
- analytics
- correlation
- forecasting
- AI explanation
- alerts
- reports
- source health

Use PostgreSQL for persistent structured data.

Use WebSockets for live UI updates.

Keep provider API keys server-side.

#### ANALYTICS

MVP:

- rolling averages
- thresholds
- rate of change
- spatial + temporal correlation

Advanced:

- Scikit-learn anomaly detection
- forecasting

- event clustering
- risk maps

## AI

Use AI only after deterministic analytics have produced structured evidence.

AI may:

- summarize current conditions
- explain an event
- generate an incident narrative
- answer grounded questions about CityPulse data

AI must never invent metrics, incidents, causes or official decisions.

## OFFICIAL COMMAND CENTER

Show:

- active events
- severity
- affected areas
- evidence
- forecast
- response resources
- assignment workflow
- incident timeline

## DESIGN

Avoid generic AI/SaaS aesthetics.

Avoid excessive glassmorphism, giant gradients, random neon borders, robot illustrations and unnecessary rounded cards.

The UI should feel like a premium civic operations system: calm, data-dense, geographic, trustworthy and modern.

Use strong typography, restrained color, map-first composition, subtle motion and clear status hierarchy. Citizen view should be simple and readable.

Official view can be information-dense.

## DIFFERENTIATION

CityPulse is not just:

- a map
- a traffic app
- an open-data catalog
- a weather dashboard
- a complaint app.

Its differentiator is cross-domain civic event fusion with evidence, explanation, forecasting and two-role workflows.

## BUILD ORDER

1. Inspect existing code.
2. Create/confirm information architecture.
3. Establish design tokens and reusable components.
4. Build data contracts and API service layer.
5. Build public dashboard.
6. Build live map.
7. Build event detail/evidence.
8. Build official command center.
9. Add analytics.
10. Add AI grounded summaries.
11. Add replay and forecasting.
12. Test responsiveness, missing feeds, loading states and error states.

Do not change backend logic merely to solve a visual issue. Do not replace working APIs with mock behavior unless explicitly requested. Preserve existing functionality when extending the project.

## 19. Recommended Build Phases

| Phase   | Focus                          | Deliverable                                                                                            |
|---------|--------------------------------|--------------------------------------------------------------------------------------------------------|
| Phase 0 | Project audit and architecture | Inspect current repo, routes, theme, components and existing APIs before changing code.                |
| Phase 1 | Data foundation                | Unified schema, city/state/area hierarchy, source registry, ingestion interfaces.                      |
| Phase 2 | MVP ingestion                  | Three strong feeds: weather + traffic + incidents/complaints. Add synthetic feeds for missing domains. |
| Phase 3 | Live public dashboard          | Map, civic health, current state, alerts and summary.                                                  |
| Phase 4 | Intelligence                   | Anomaly rules, spatial-temporal correlation and event clustering.                                      |
| Phase 5 | Official command center        | Priority incidents, evidence, response workflow.                                                       |
| Phase 6 | AI                             | Grounded explanation and incident narrative.                                                           |
| Phase 7 | Advanced demo                  | Forecasting, historical replay, risk map, source health.                                               |

## 20. Judge Demo Story

6. Start with a normal city state and show the unified dashboard.
7. Open the live map and show the data layers.
8. Trigger a simulated heavy-rain event in one zone.
9. Show traffic congestion increasing in the same zone.
10. Show flood complaints increasing shortly afterward.
11. Show transit delays appearing in the same area.
12. CityPulse creates one multi-signal event cluster.
13. Open the evidence panel and show timestamps, locations and source types.
14. Show the plain-language AI explanation.
15. Show the forecast/risk view.
16. Switch to the official Command Center and show the prioritized incident.
17. Open Historical Replay and replay the same event from beginning to peak.

## 21. 24-Hour MVP vs Advanced Ceiling

| MUST HAVE                | SHOULD HAVE             | ADVANCED                   |
|--------------------------|-------------------------|----------------------------|
| 3 data feeds             | 5–8 data categories     | Kafka streaming            |
| Normalized schema        | Source health           | ML forecasting             |
| Live map                 | Citizen reporting       | Agentic monitoring         |
| Anomaly/correlation rule | Historical replay       | Advanced digital twin      |
| Plain-language summary   | Official command center | Resource optimization      |
| Event detail + evidence  | Risk map                | Advanced predictive models |

## 22. Major Risks and Mitigations

- Too many data sources: start with three strongly connected feeds and expand only after the fusion works.
- API availability: build synthetic connectors with the same schema as live connectors.
- Data mismatch: normalize every source into a common event contract.
- False correlations: require spatial + temporal overlap and show evidence.
- AI hallucination: generate summaries only from a structured evidence bundle.
- Map complexity: use layer toggles and progressive disclosure.
- Kafka overengineering: defer Kafka until the MVP works.
- Fake real-time claims: label simulated data and show timestamps/freshness.

## 23. Research References

**UrbanLogiq** — Government-focused AI-powered data integration, cross-domain analytics and predictive decision intelligence. <https://urbanlogiq.com/>

**UrbanLogiq Data Platform** — Shared ontology, ingestion, normalization, modeling and governance. <https://urbanlogiq.com/data-platform>

**ArcGIS Hub** — Open data, collaboration, public engagement and geographic applications. <https://www.esri.com/en-us/arcgis/products/arcgis-hub/overview>

**ArcGIS Dashboards** — Real-time location-based dashboards and operational monitoring. <https://www.esri.com/en-us/arcgis/products/arcgis-dashboards/overview>

**Waze for Cities** — Traffic incidents, closures, city-to-driver communication and mobility analytics. <https://www.waze.com/wazeformcities/>

**urbanOS CityPulse** — Existing product using the CityPulse name for visitor analytics, forecasts and loudness. <https://urbanos.de/en/apps/city-pulse>

**Socrata Developers** — Government open-data API ecosystem. <https://dev.socrata.com/>

## 24. Final Strategy

The strongest CityPulse strategy is not to compete on the number of APIs or the prettiness of the map. Build a working intelligence loop around one compelling event. For the hackathon demo, use a weather → traffic → flooding/transit chain because it is easy for judges to understand visually and demonstrates the core requirement: meaningful fusion of heterogeneous feeds.

The product should feel different from an ordinary smart-city dashboard because it does not stop at showing data. It identifies a developing multi-signal situation, shows the evidence behind it, explains it in plain language, estimates what may happen next, and presents the information differently for citizens and operators.

**One-line pitch:** “CityPulse turns disconnected city signals into a live, explainable picture of what is changing, where it is changing, and what may need attention next.”

*Planning document only — no project implementation was performed.*